

Rapport de mise en production

Création d'une organisation comptable sur le Kernel YOWYOB
(owner, organisation, services, utilisateurs et rôles)

ALD

azangueleonel9@gmail.com

17 juin 2026 — environnement `kernel-core.yowyob.com`

Résumé

Ce rapport documente, de bout en bout, la création en **production** d'une organisation comptable (KSM Comptabilité) sur la plateforme YOWYOB : création du *owner*, vérification d'e-mail, création et approbation de l'organisation, souscription des services, puis création des utilisateurs comptables et affectation de leurs rôles. Chaque étape est accompagnée de la commande `curl` réellement exécutée, du résultat obtenu, ainsi que des **problèmes rencontrés** et de leur résolution. Toutes les opérations ont été réalisées en *self-service* avec le compte owner (jeton portant `tenant:admin`), sans recourir au `platform-admin`. Le rapport documente enfin le **flux de connexion multi-contexte** (*identify / discover-contexts / select-context*) implémenté pour rendre la plateforme multi-tenant : le Kernel résout lui-même le tenant à partir du seul couple e-mail / mot de passe, en remplacement du tenant codé en dur côté front.

Table des matières

1	Environnement et prérequis	2
1.1	Points d'accès	2
1.2	Identifiants client (X-Client-Id / X-API-Key)	2
2	Création du owner	2
2.1	Sign-up	2
2.2	Vérification de l'e-mail	3
3	Connexion du owner	3
4	Création et approbation de l'organisation	4
4.1	Création	4
4.2	Approbation (self-approve)	4
5	Souscription des services	5
6	Création des utilisateurs (membres comptables)	5
6.1	Principe : <code>register</code> , pas <code>sign-up</code>	5
6.2	Login d'un compte non encore vérifié	6
7	Création des rôles et affectations	6
7.1	Création des rôles dans le tenant owner	6
7.2	Affectation des rôles aux utilisateurs	7
7.3	Matrice des droits comptables (backend)	7

8 Flux de connexion multi-contexte (identity / context)	7
8.1 Principe : le Kernel résout lui-même les tenants	7
8.2 Backend — proxy d'authentification (les clés restent côté serveur)	8
8.3 Frontend — écran de sélection d'espace	8
9 Validation de bout en bout	9
10 Problèmes rencontrés — synthèse	9
11 Synthèse des identifiants de production	10

1 Environnement et prérequis

1.1 Points d'accès

Rôle	URL
Kernel (API centrale)	https://kernel-core.yowyob.com
Backend accounting (ERP)	https://accounting.yowyob.com/accounting-api
Frontend	https://accounting.yowyob.com

1.2 Identifiants client (X-Client-Id / X-API-Key)

Le Kernel exige, sur les routes d'authentification, un couple **X-Client-Id / X-API-Key** identifiant l'application appelante. Pour toutes les opérations métier (sign-up, login, organisation, rôles) le client utilisé est **accounting-backend**.

```
# Variables réutilisées dans tout le rapport
export KERNEL=https://kernel-core.yowyob.com
export BACKEND=https://accounting.yowyob.com/accounting-api

# Client applicatif "accounting-backend"
export CID=*****
export KEY=XXXXXXXXX
```

Piège n°1 — le tenant plateforme n'est pas le tenant de l'utilisateur

Le tenant 11111111-1111-1111-1111-111111111111 est le tenant *plateforme* (réservé au platform-admin et à l'autorisation des clients). Chaque utilisateur créé reçoit en revanche **son propre tenant**, généré par le Kernel. Toute la suite repose sur le tenant de l'owner, et non sur le tenant plateforme.

2 Création du owner

2.1 Sign-up

Le compte propriétaire est créé via POST /api/auth/sign-up avec accountType=BUSINESS. Le Kernel répond 201 et renvoie déjà un accessToken (TTL 900s) portant ROLE_OWNER, tenant:admin, organizations:write, accounting:*, treasury:*

```
curl -X POST "$KERNEL/api/auth/sign-up" \
  -H "Content-Type: application/json" \
  -H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
  -d '{
    "firstName": "Leonel",
```

```
"lastName":"Azangue",
"email":"azangueleonel9@gmail.com",
"password":"Azangue!Owner2026",
"accountType":"BUSINESS"
},'
```

Réponse — HTTP 201

```
{"success":true,"data":{"id":"f3c5743d-ee96-4349-abf2-5fb13dc89a43",
"tenantId":"4b78ea0c-11b6-4b01-ad03-187c890d5510", "actorId":"2c3e4c11-7ddd-4e84-936d-1f15f7ac84cc",
"email":"azangueleonel9@gmail.com", "emailVerified":false, ... ,"accessToken":"eyJ...", ...},
"message":"User signed up."}
```

Identifiants obtenus.

Champ	Valeur
userId	f3c5743d-ee96-4349-abf2-5fb13dc89a43
tenantId (owner)	4b78ea0c-11b6-4b01-ad03-187c890d5510
actorId	2c3e4c11-7ddd-4e84-936d-1f15f7ac84cc
mot de passe	Azangue!Owner2026

2.2 Vérification de l'e-mail

L'e-mail de confirmation (SMTP) contient un jeton JWT (typ = auth-email-verification, TTL 86 400 s). La confirmation se fait par API.

Piège n°2 — nom exact du champ

Le corps doit contenir le champ `verificationToken` (et **pas** `token`, ni `code`, ni `emailVerificationToken`, qui renvoient tous 400). Endpoint : POST `/api/auth/email-verification/confirm`, sans Bearer (juste le couple client/clé).

```
curl -X POST "$KERNEL/api/auth/email-verification/confirm" \
-H "Content-Type: application/json" \
-H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
-d '{"verificationToken":"eyJ...<jwt reçu par email>..."}'
```

Réponse — HTTP 200

```
{"success":true,"data":{"...,"emailVerified":true, "emailVerifiedAt":"2026-06-17T19:12:58Z"...},"message":"Email verified."}
```

Problème rencontré — SMTP muet, résolu

Lors d'une première tentative, l'e-mail de vérification n'arrivait jamais (SMTP de production muet) et le `platform-admin` était inutilisable (code MFA envoyé sur une boîte sans accès). Après intervention du DevOps, le compte a été **recréé avec les mêmes informations** : l'e-mail a alors été reçu et la confirmation a réussi. À noter que le Kernel autorise malgré tout le *login* d'un compte non encore vérifié (cf. §6.2).

3 Connexion du owner

Le login renvoie un `accessToken` (900 s) utilisé pour toutes les opérations administratives suivantes.

Piège n°3 — le tenant est obligatoire au login

Le login Kernel exige l'en-tête `X-Tenant-Id` = **tenant de l'owner** (4b78ea0c...). Avec le tenant plateforme 11111111..., l'authentification échoue (401).

```
export OWNER_TENANT=4b78ea0c-11b6-4b01-ad03-187c890d5510

TOKEN=$(curl -s -X POST "$KERNEL/api/auth/login" \
-H "Content-Type: application/json" \
-H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
-H "X-Tenant-Id: $OWNER_TENANT" \
-d '{"principal":"azangueleone19@gmail.com","password":"Azangue!Owner2026"}' \
| grep -o '"accessToken": "[^"]*"' | head -1 | cut -d'"' -f4)
```

4 Création et approbation de l'organisation

4.1 Création

```
ORG=$(curl -s -X POST "$KERNEL/api/organizations" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
-H "X-Tenant-Id: $OWNER_TENANT" \
-d '{
  "businessActorId": "2c3e4c11-7ddd-4e84-936d-1f15f7ac84cc",
  "code": "KSM-CPTA",
  "organizationType": "ENTERPRISE",
  "displayName": "KSM Comptabilité",
  "legalName": "KSM Comptabilité"
}')
```

Réponse — HTTP 200

```
{
  "success": true,
  "data": {
    "id": "b45f1edb-a269-4ae2-b2d1-a4a057c7b3c9",
    "code": "KSM-CPTA",
    "governanceStatus": "PENDING_APPROVAL",
    ...
  },
  "message": "Organization created."
}
```

`orgId = b45f1edb-a269-4ae2-b2d1-a4a057c7b3c9`.

4.2 Approbation (self-approve)

L'owner possède `tenant:admin`, ce qui satisfait la garde `canGovernOrganizations` : il peut approuver lui-même.

Piège n°4 — le champ `reason` est obligatoire

`POST /api/organizations/{id}/approve` sans corps renvoie 400. Il faut fournir `{"reason": "..."}.`

```
curl -X POST "$KERNEL/api/organizations/$ORG_ID/approve" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
-H "X-Tenant-Id: $OWNER_TENANT" \
-d '{"reason": "Owner self-approval"}'
```

Réponse — HTTP 200

```
{
  "success": true,
  "data": {
    ...,
    "governanceStatus": "APPROVED",
    "governedByUserId": "f3c5743d-...",
    "status": "APPROVED"
  },
  "message": "Organization approved."
}
```

5 Souscription des services

Chaque service est souscrit via POST `/api/organizations/{id}/services`.

Piège n°5 — dépendances entre services

TREASURY exige BANKING au préalable (*it cannot subscribe TREASURY without required services BANKING*). BILLING recommande (sans bloquer) ACCOUNTING, TREASURY et CASHIER. L'ordre retenu : COMMERCIAL, BILLING, ACCOUNTING, BANKING, TREASURY, CASHIER.

```
for svc in COMMERCIAL BILLING ACCOUNTING BANKING TREASURY CASHIER; do
  curl -s -X POST "$KERNEL/api/organizations/$ORG_ID/services" \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer $TOKEN" \
    -H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
    -H "X-Tenant-Id: $OWNER_TENANT" \
    -d '{"serviceCode":"$svc"}'
done
```

Résultat

Tous HTTP 200. `effectiveServices` final : [ORGANIZATION, SETTINGS, COMMERCIAL, BILLING, ACCOUNTING, BANKING, TREASURY, CASHIER], `dependencyIssues`: [].

6 Création des utilisateurs (membres comptables)

6.1 Principe : register, pas sign-up

Piège n°6 — sign-up créerait un nouveau tenant

POST `/api/auth/sign-up` génère **un nouveau tenant** à chaque appel. Pour que les membres **partagent le tenant de l'owner**, il faut utiliser POST `/api/auth/register` avec le Bearer de l'owner et X-Tenant-Id = tenant owner. La garde côté Kernel est `canManageIdentity` (l'owner l'a via `tenant:admin`).

Piège n°7 — DTO de register

Le corps attendu est {username, email, password, authProvider}. Envoyer `firstName/lastName/accountType` renvoie 400. `register` ne crée **pas** d'*actor* (`actorId:null`), ce qui n'empêche ni le login ni l'usage des rôles.

```
# En-têtes communs owner
H_OWNER=(-H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
  -H "X-Tenant-Id: $OWNER_TENANT")

# 1) Responsable comptable
curl -X POST "$KERNEL/api/auth/register" "${H_OWNER[@]}" \
  -d '{"username":"leonel.azangue",
    "email":"leonel.azangue@facsciences-uy1.cm",
    "password":"Leonel!Compta2026","authProvider":"LOCAL"}'

# 2) Comptable
curl -X POST "$KERNEL/api/auth/register" "${H_OWNER[@]}" \
  -d '{"username":"balaandegue",
    "email":"balaandeguefrancoislionnel@gmail.com",
    "password":"Balla!Compta2026","authProvider":"LOCAL"}'

# 3) Aide comptable
```

```
curl -X POST "$KERNEL/api/auth/register" "${H_OWNER[@]}" \
-d '{"username":"ocali597198",
    "email":"ocali597198@gmail.com",
    "password":"Ocali!Compta2026","authProvider":"LOCAL"}'
```

Résultat — HTTP 201 pour chaque membre

Chaque réponse : "tenantId":"4b78ea0c-..." (= tenant owner, partagé), "actorId":null, "emailVerified":false, "message":"User registered."

Identifiants des membres.

userId	e-mail / mot de passe	rôle visé
a83056d8-9211-4efc-b815-1e10f5769a6	leonel.azangue@facsciences-uy1.cm Leonel!Compta2026	RESPONSABLE_COMPTABLE
756f407f-b961-4f95-a3d0-ba6cb2743842	balla@coislionnel@gmail.com / Balla!Compta2026	COMPTABLE
b126232f-963e-4b67-9f9c-6eab5071b80b	ocali597198@gmail.com Ocali!Compta2026	AIDE_COMPTABLE

6.2 Login d'un compte non encore vérifié

```
curl -X POST "$KERNEL/api/auth/login" \
-H "Content-Type: application/json" \
-H "X-Client-Id: $CID" -H "X-API-Key: $KEY" \
-H "X-Tenant-Id: $OWNER_TENANT" \
-d '{"principal":"leonel.azangue@facsciences-uy1.cm",
    "password":"Leonel!Compta2026"}'
```

Réponse — HTTP 200

```
{"success":true,"data":{...,"emailVerified":false...}, "message":"User authenticated."}
```

Le Kernel **autorise le login** malgré emailVerified:false : la vérification d'e-mail n'est donc *pas bloquante* pour rendre les membres opérationnels.

7 Création des rôles et affectations

7.1 Création des rôles dans le tenant owner

Les rôles n'existent que par tenant. Dans le tenant owner seul OWNER préexistait ; les trois rôles comptables sont donc créés.

Correspondance permission → autorité backend

Le backend comptable utilise hasRole('COMPTABLE'), qui teste l'autorité ROLE_COMPTABLE. Le JwtAuthenticationFilter préfixe ROLE_ à la chaîne brute du claim permissions. Il suffit donc que la permission du rôle soit **son propre code** (ex. COMPTABLE).

```
for code in RESPONSABLE_COMPTABLE COMPTABLE AIDE_COMPTABLE; do
  curl -s -X POST "$KERNEL/api/roles" "${H_OWNER[@]}" \
  -d '{"code":"$code","name":"$code",
      "scopeType":"TENANT","permissions":["$code"]}'
done
```

Résultat — HTTP 200/201, "Role created."

```
RESPONSABLE_COMPTABLE → roleId 0149ada3-9c33-4ff1-ac32-f4ebd24493dd
COMPTABLE → roleId ec4883bf-95a1-488e-8eb4-0bce48810d55
AIDE_COMPTABLE → roleId e2cb762c-69b5-42c2-876b-b577b5591473
```

7.2 Affectation des rôles aux utilisateurs

POST /api/roles/assignments avec scope=TENANT.

```
# Exemple : COMPTABLE -> balaandegue
curl -X POST "$KERNEL/api/roles/assignments" "${H_OWNER[@]}" \
-d '{
  "userId": "756f407f-b961-4f95-a3d0-1aacb27138a2",
  "roleId": "ec4883bf-95a1-488e-8eb4-0bce48810d55",
  "scopeType": "TENANT",
  "scopeId": "4b78ea0c-11b6-4b01-ad03-187c890d5510",
  "scope": "TENANT"
}'
```

Résultat — HTTP 201 pour les 3 affectations

```
"message": "Role assigned." (leonel → RESPONSABLE_COMPTABLE, balaandegue → COMPTABLE,
ocali → AIDE_COMPTABLE).
```

7.3 Matrice des droits comptables (backend)

Capacité	AIDE	COMPTABLE	RESPONSABLE	ADMIN
Lecture / consultation	✓	✓	✓	✓
Saisie de brouillons	✓	✓	✓	✓
Gestion (comptes, jour- naux, taxes, validation)	✗	✓	✓	✓
Clôtures, paramètres, pé- riodes	✗	✗	✓	✓
Opérations techniques (synchro, purge)	✗	✗	✗	✓

8 Flux de connexion multi-contexte (identity / context)

Jusqu'ici le login simple (§3) imposait au front de **connaître le tenant à l'avance** (en-tête X-Tenant-Id), ce que le frontend faisait en codant en dur NEXT_PUBLIC_TENANT_ID. C'est mono-tenant par construction : injecté au build, ce tenant est identique pour tous les visiteurs et épingle tout le déploiement à une seule organisation. Pour rendre la plateforme réellement multi-tenant / multi-organisation, nous avons **implémenté** le flux de connexion contextuel natif du Kernel, proxifié de bout en bout par le backend.

8.1 Principe : le Kernel résout lui-même les tenants

Le compte n'envoie plus que son **email + password**. Le Kernel identifie tous les contextes (tenants) auxquels le principal appartient, ainsi que les organisations accessibles dans chacun. L'utilisateur choisit ensuite son espace ; le tenant et l'organisation retenus sont renvoyés au front, qui les stocke en session et les rejoue en X-Tenant-Id / X-Organization-Id.

Étape (endpoint Kernel)	Rôle
POST /api/auth/identify	(optionnel) {principal} → {accountExists, nextStep, matchingAccountCount}. Indique si le compte existe et l'étape suivante.
POST /api/auth/discover-contexts	{principal, password} → {selectionToken, expiresInSeconds, contexts[]}. C'est ici qu'on découvre le(s) tenant(s) sans les connaître à l'avance : chaque contexte = {contextId, tenantId, userId, actorId, organizations[]}. C'est ici qu'on découvre le(s) tenant(s) sans les connaître à l'avance : chaque contexte = {contextId, tenantId, userId, actorId, organizations[]}.
POST /api/auth/select-context	{selectionToken, contextId, organizationId?} → {selectedTenantId, selectedOrganizationId, session}. Finalise le login pour le contexte (et l'organisation) choisi.

8.2 Backend — proxy d'authentification (les clés restent côté serveur)

Le backend (config/auth/AuthController.java + AuthService.java) expose trois endpoints jumeaux sur /api/auth/* qui **proxient** le Kernel. À chaque appel, addClientCredentials injecte X-Client-Id / X-API-Key côté serveur : le navigateur ne voit jamais ni les clés Kernel ni de tenant codé en dur.

Endpoint backend	Délègue à (Kernel) / corps
POST /api/auth/identify	/api/auth/identify {principal}
POST /api/auth/discover-contexts	/api/auth/discover-contexts {email, password}
POST /api/auth/select-context	/api/auth/select-context {selectionToken, contextId, organizationId?}

Le DTO LoginResponse a reçu un champ tenantId; parseKernelContextualLoginResponse lit data.{selectedTenantId, selectedOrganizationId, session} et reporte le tenant/organisation effectivement choisis dans la réponse renvoyée au front.

```
# Étape 1 découverte des contextes (aucun tenant, aucune clé côté navigateur)
curl -s -X POST "$BACKEND/api/auth/discover-contexts" \
-H "Content-Type: application/json" \
-d '{"email":"azangueleonel9@gmail.com","password":"Azangue!Owner2026"}'

# Étape 2 sélection du contexte (+ organisation si plusieurs) -> {token, user, tenantId}
curl -s -X POST "$BACKEND/api/auth/select-context" \
-H "Content-Type: application/json" \
-d '{"selectionToken":"<reçu à l'étape 1>",
  "contextId":"<contexte choisi>",
  "organizationId":"b45f1edb-a269-4ae2-b2d1-a4a057c7b3c9"}'
```

8.3 Frontend — écran de sélection d'espace

components/landing/login-modal.tsx n'envoie plus de tenantId. handleLogin appelle discover-contexts, aplatit les contextes en options (buildOptions : une option par organisation, ou 1 Espace personnel si aucune org). S'il n'y a **qu'une** option, le login est finalisé automatiquement; sinon un **écran de sélection** liste les espaces (boutons par organisation/tenant). completeSelection appelle select-context puis persiste en localStorage : tenant_id = response.tenantId || option.tenantId || env, et organization_id = option.organizationId || user.organizationId || env (le NEXT_PUBLIC_* ne sert plus que de dernier repli mono-client).

Validation live (2026-06-17) — owner azangueleone19

discover-contexts renvoie 1 contexte sur le tenant 4b78ea0c... avec organizations:[] (vide)
; select-context renvoie selectedOrganizationId:null + session OK ⇒ login finalisé, tenant résolu dynamiquement.

Trou de données à combler — multi-org

organizations[] est alimenté par l'appartenance à l'organisation (UserOrganizationAccess). Or les rôles affectés au scope TENANT (§7, membres comptables) ne créent **pas** cette appartenance : le sélecteur d'organisation ressort donc *vide*. Le tenant, lui, est déjà géré dynamiquement. Pour un vrai multi-org, il faudra rattacher chaque utilisateur à l'organisation (*membership*) en plus des rôles tenant.

État de livraison

Flux **implémenté** (backend config/auth + front login-modal.tsx), compilé (backend mvn compile OK ; tsc clean sur login-modal) et validé en local. **Pas encore commité ni déployé en prod** : son activation en production nécessite un redéploiement du backend et du frontend.

9 Validation de bout en bout

Connexion d'un membre **via le backend** (exactement comme le frontend), puis appel d'un endpoint comptable réel sur l'organisation KSM-CPTA.

```
# 1) Login via le backend accounting (contrat {token, user})
TOKEN_M=$(curl -s -X POST "$BACKEND/api/auth/login" \
-H "Content-Type: application/json" \
-d '{"email":"balaandeguefrancoislionnel@gmail.com",
      "password":"Balla!Compta2026",
      "tenantId":"4b78ea0c-11b6-4b01-ad03-187c890d5510"}' \
| grep -o '"token": "[^"]*"' | cut -d'"' -f4)

# 2) Appel d'un endpoint comptable (rôle COMPTABLE + org KSM-CPTA)
curl "$BACKEND/api/accounting/journals" \
-H "Authorization: Bearer $TOKEN_M" \
-H "X-Tenant-Id: 4b78ea0c-11b6-4b01-ad03-187c890d5510" \
-H "X-Organization-Id: b45f1edb-a269-4ae2-b2d1-a4a057c7b3c9"
```

Réponse — HTTP 200

```
{"success":true,"message":"Journals list retrieved","data":[],"code":200}
Claim permissions du JWT : [COMPTABLE, COMPTABLE#TENANT, ROLE_COMPTABLE, ROLE_COMPTABLE#TENANT]
```

La chaîne complète frontend → backend → Kernel est donc fonctionnelle : authentification, propagation du tenant, rôle reconnu, et autorisation sur l'organisation.

10 Problèmes rencontrés — synthèse

Problème	Cause	Résolution
E-mail de vérification jamais reçu	SMTP de production muet ; platform-admin inutilisable (MFA hors d'accès)	Intervention DevOps + re-création du compte <i>mêmes infos</i> ; e-mail reçu

Login front → 401 puis 502	Le front envoie un <code>tenantId</code> codé en dur 11111111... (tenant plateforme), alors que l'utilisateur appartient à son propre tenant. Les essais répétés déclenchent le throttle Kernel (429 AUTH_THROTTLED_PRINCIPAL), que le backend traduit en 502.	Solution pérenne : flux <i>identity/context</i> (ğ8) où le Kernel résout le tenant, à la place du <code>NEXT_PUBLIC_TENANT_ID</code> codé en dur; attendre la retombée du throttle
approve renvoie 400	Champ <code>reason</code> manquant	Ajouter <code>{"reason": "..."} </code>
TREASURY refusé 400	Dépendance BANKING non souscrite	Souscrire BANKING d'abord
register renvoie 400	Mauvais DTO (firstName/lastName/ accountType)	Corps <code>{username,email,password,authProvider}</code>
verification/confirm renvoie 400	Mauvaise clé de corps	Utiliser <code>verificationToken</code>

11 Synthèse des identifiants de production

Élément	Valeur
Tenant (owner, partagé)	4b78ea0c-11b6-4b01-ad03-187c890d5510
Organisation KSM-CPTA	b45f1edb-a269-4ae2-b2d1-a4a057c7b3c9
Owner	azangueleonel9@gmail.com / Azangue!Owner2026
Responsable comptable	leonel.azangue@facsciences-uy1.cm / Leonel!Compta2026
Comptable	balaandeguefrancoislionnel@gmail.com / Balla!Compta2026
Aide comptable	ocali597198@gmail.com / Ocali!Compta2026
Services actifs	COMMERCIAL, BILLING, ACCOUNTING, BANKING, TREASURY, CASHIER

Conclusion sur le partage tenant / organisation. Les utilisateurs créés via **register partagent le tenant** de l'owner. L'organisation, elle, n'est pas une appartenance gravée dans le jeton (`organizations:[]` est vide même pour l'owner) : le partage de l'organisation se matérialise par l'envoi du **même en-tête X-Organization-Id** (b45f1edb...) sur les requêtes métier. Le frontend retombe sur `NEXT_PUBLIC_ORGANIZATION_ID` lorsque `user.organizationId` est vide.